

Model Efficiency

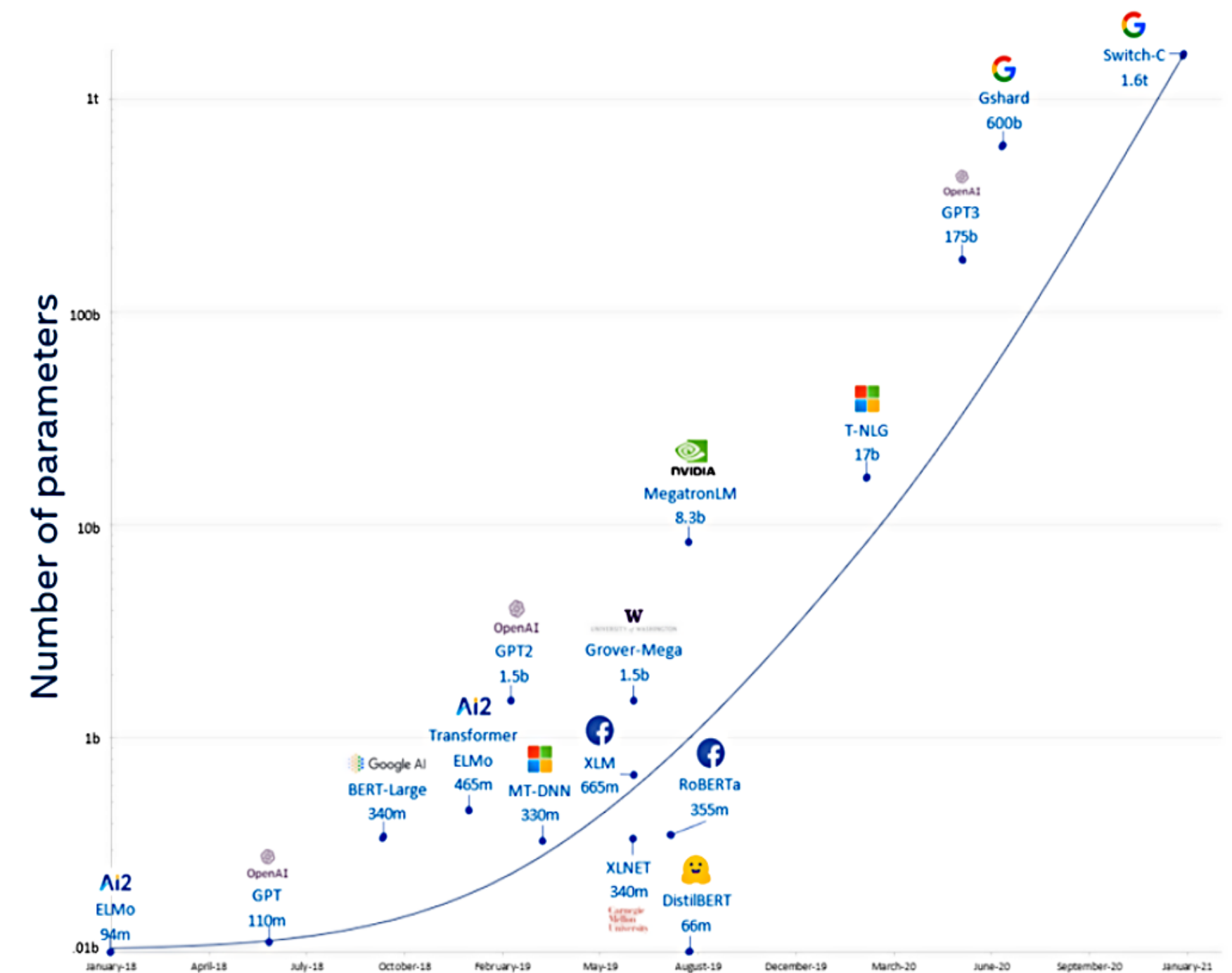
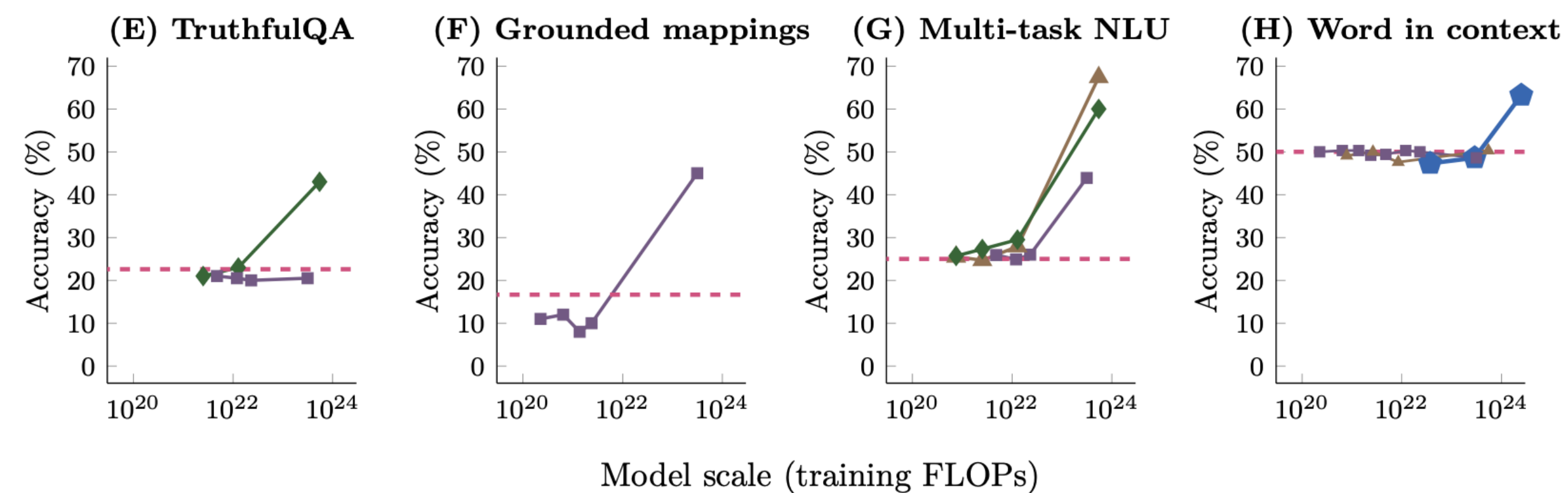
Antoine Bosselut

Outline

- Motivation
- Model Compression methods
 - Pruning
 - Quantization
 - Knowledge distillation
- Inference Efficiency
 - KV Cache
 - Speculative Decoding

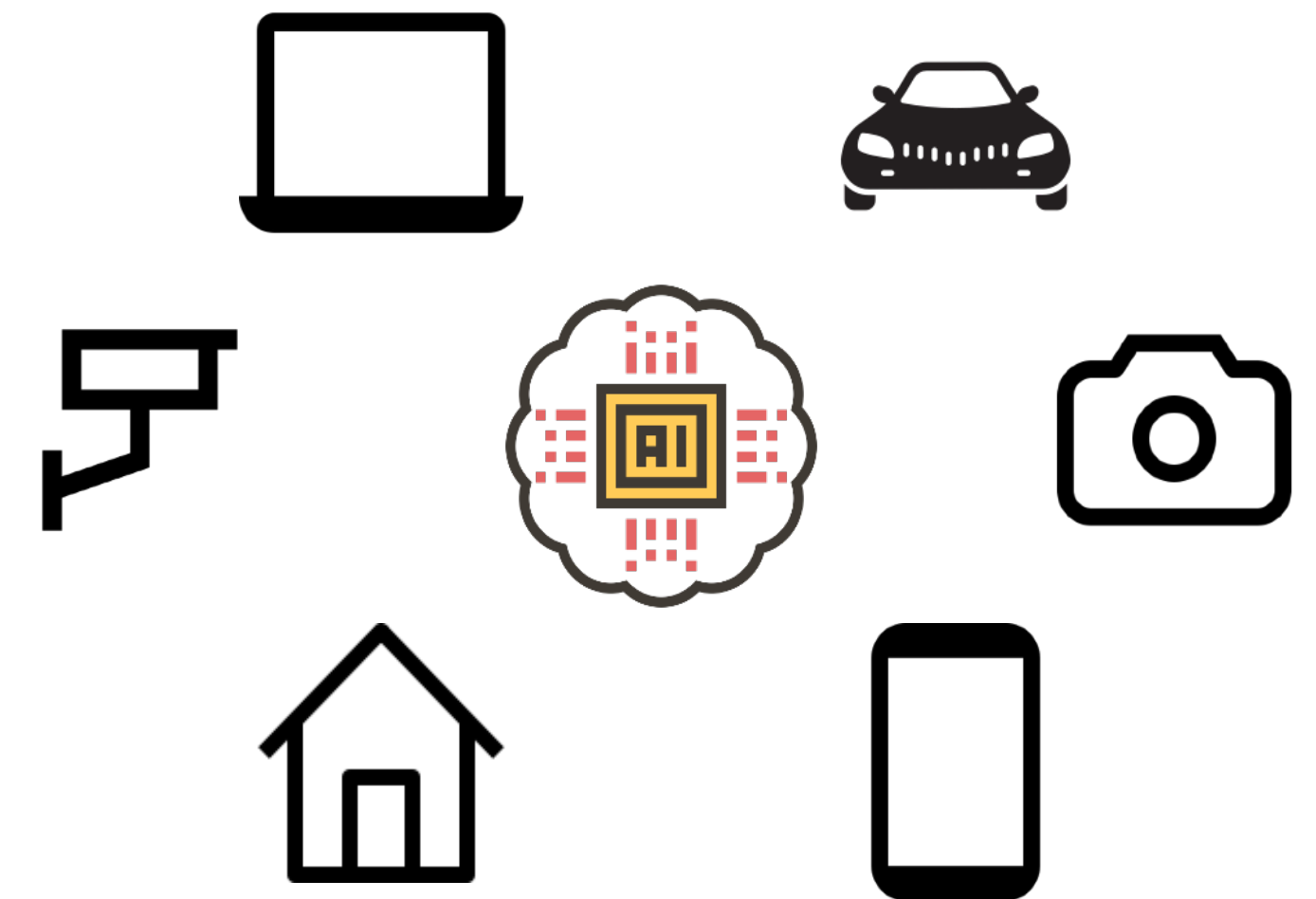
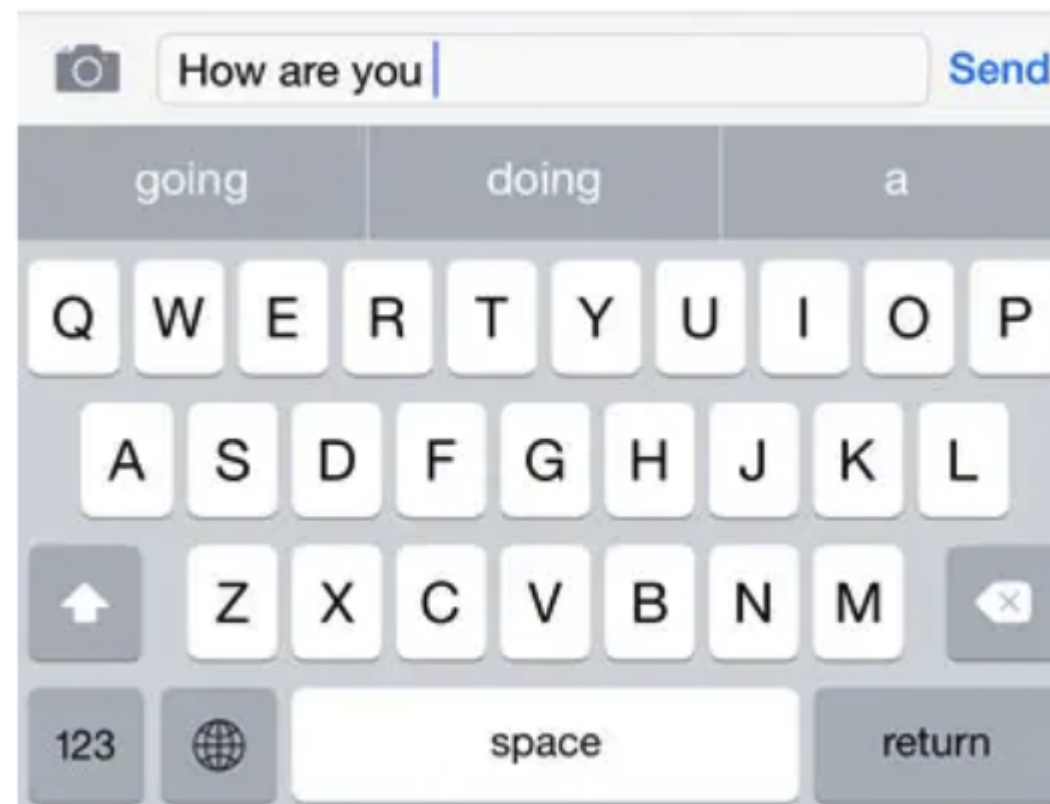
Growth of model parameters

- Exponential growth in model parameters
 - Scaling laws
 - Emergent abilities of LLMs



LLM Deployment in Production

- Cloud processing not always possible
 - Latency issue
 - Data privacy
- Inference time for edge devices
- Memory issue
 - ~350 GB just for **storing** LLM weights!
- Finetuning LLMs
 - Time-consuming
 - Expensive



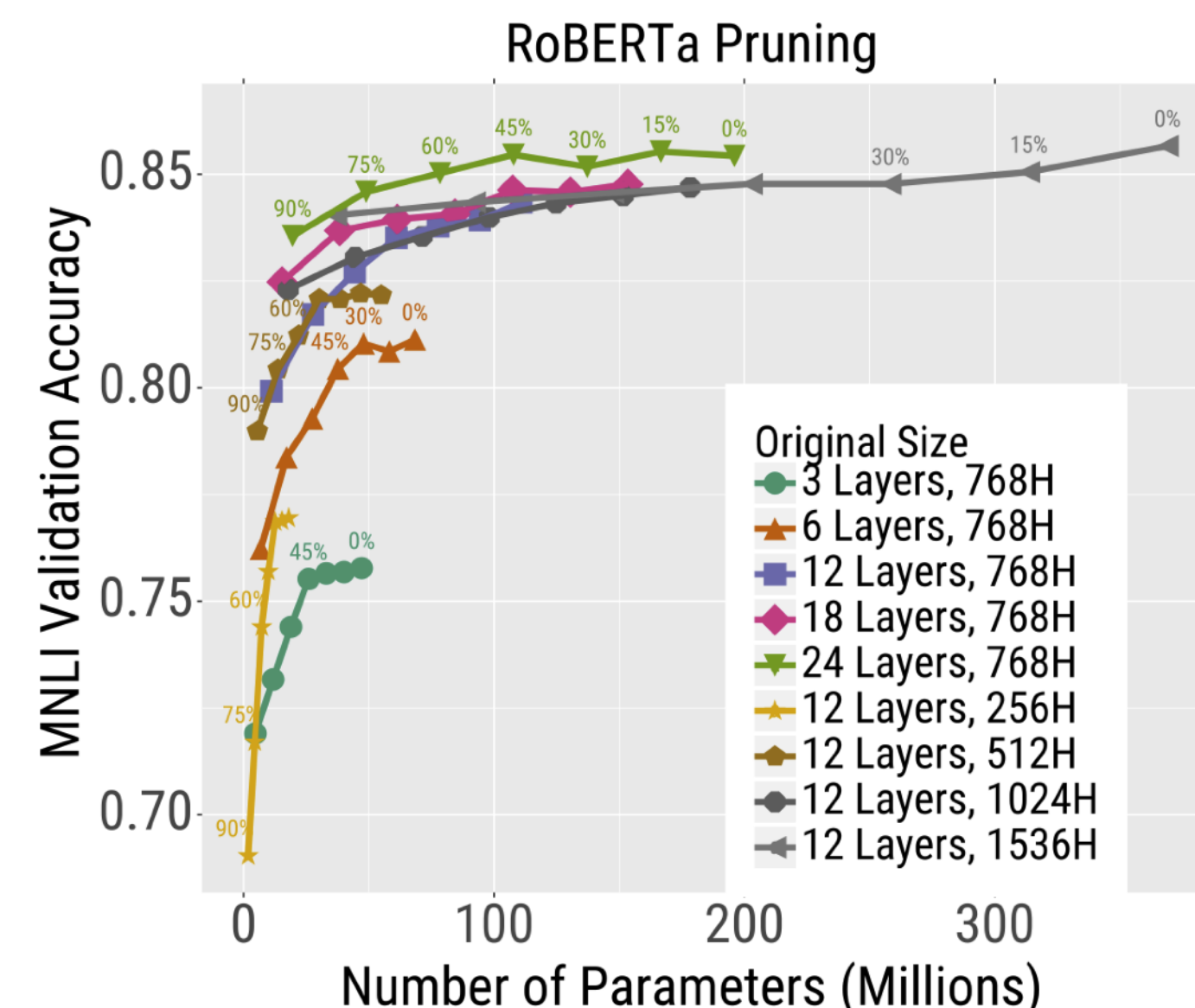
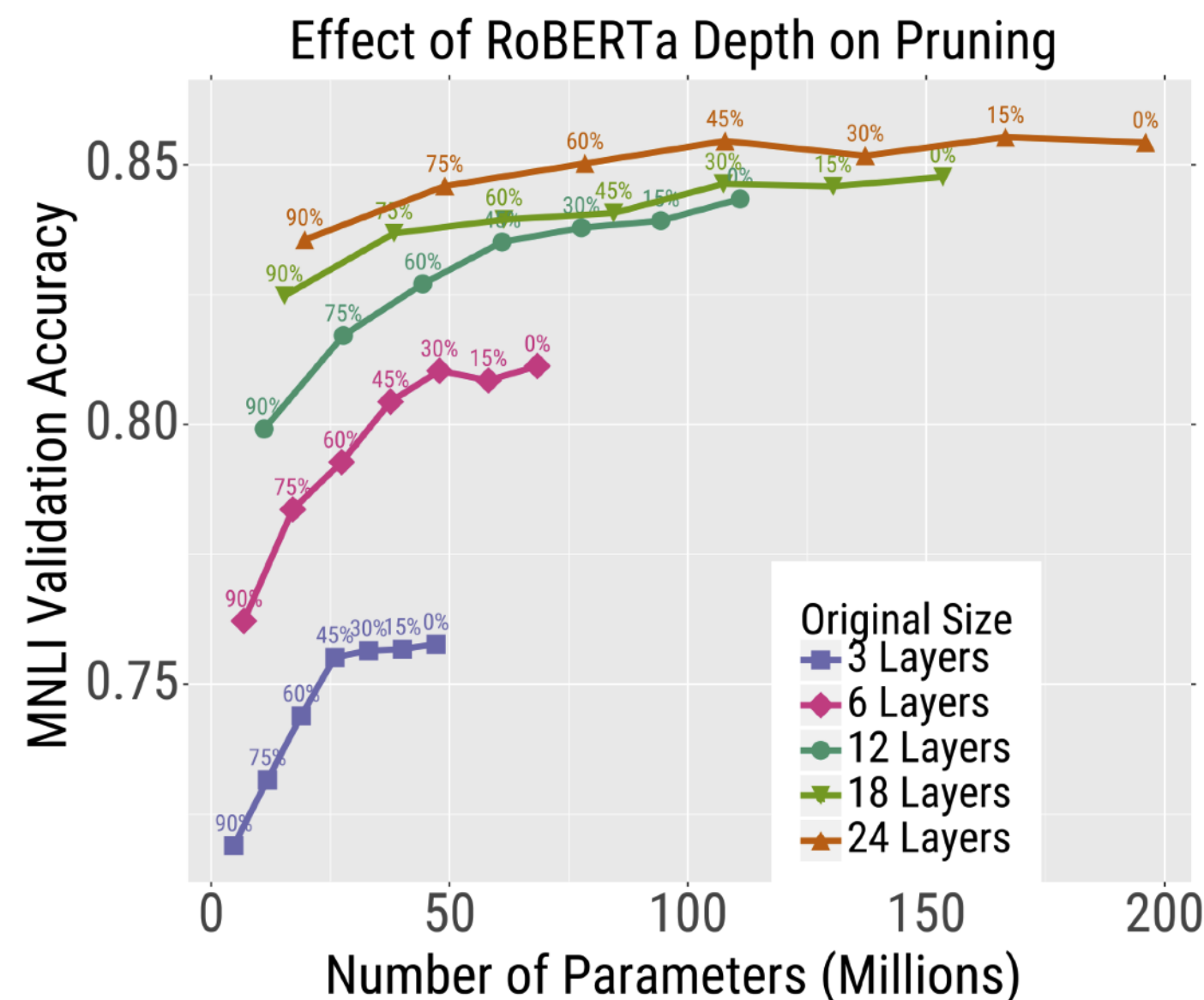
What can we do instead?

Train smaller models!

Compression can reduce inference cost of deploying models!

Train Large, then Compress!

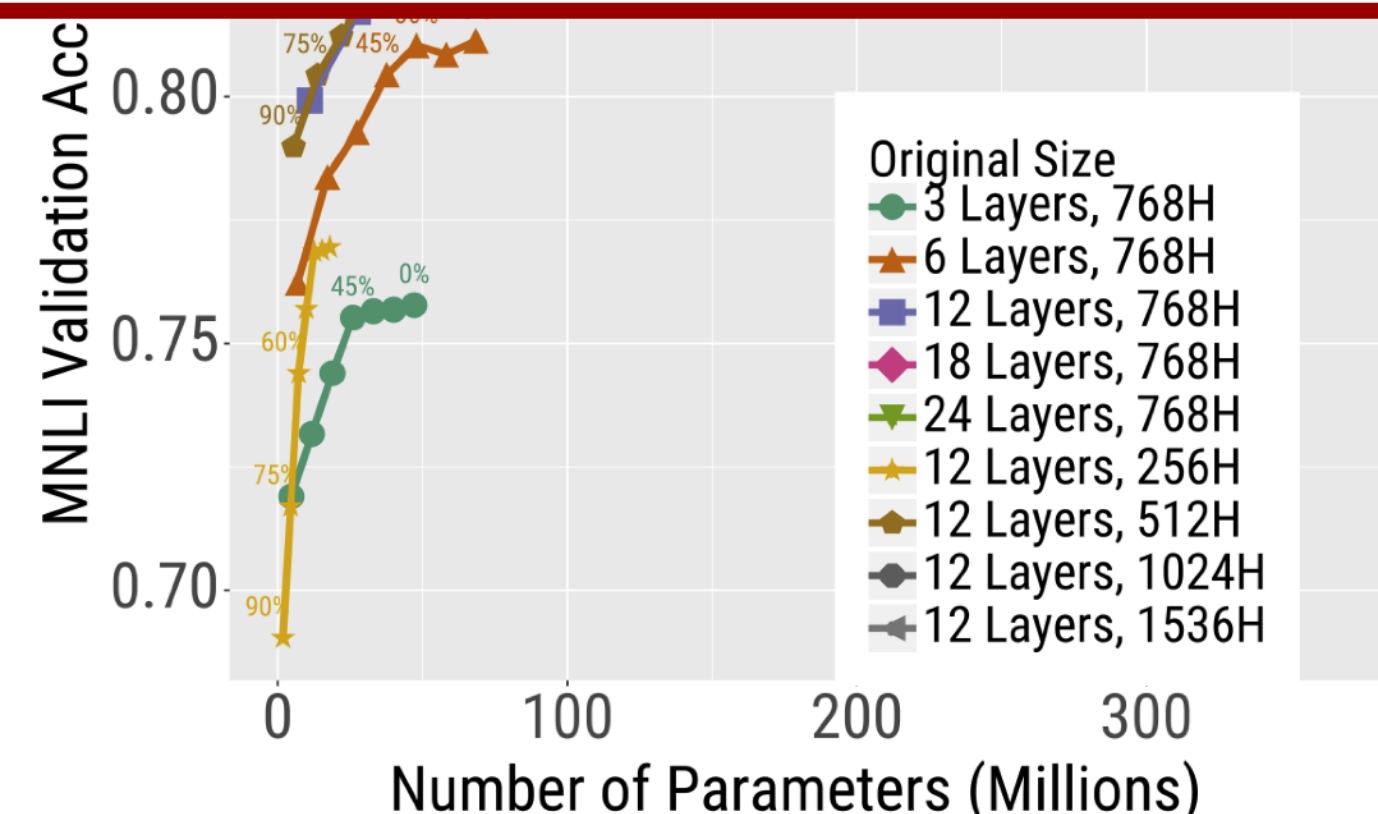
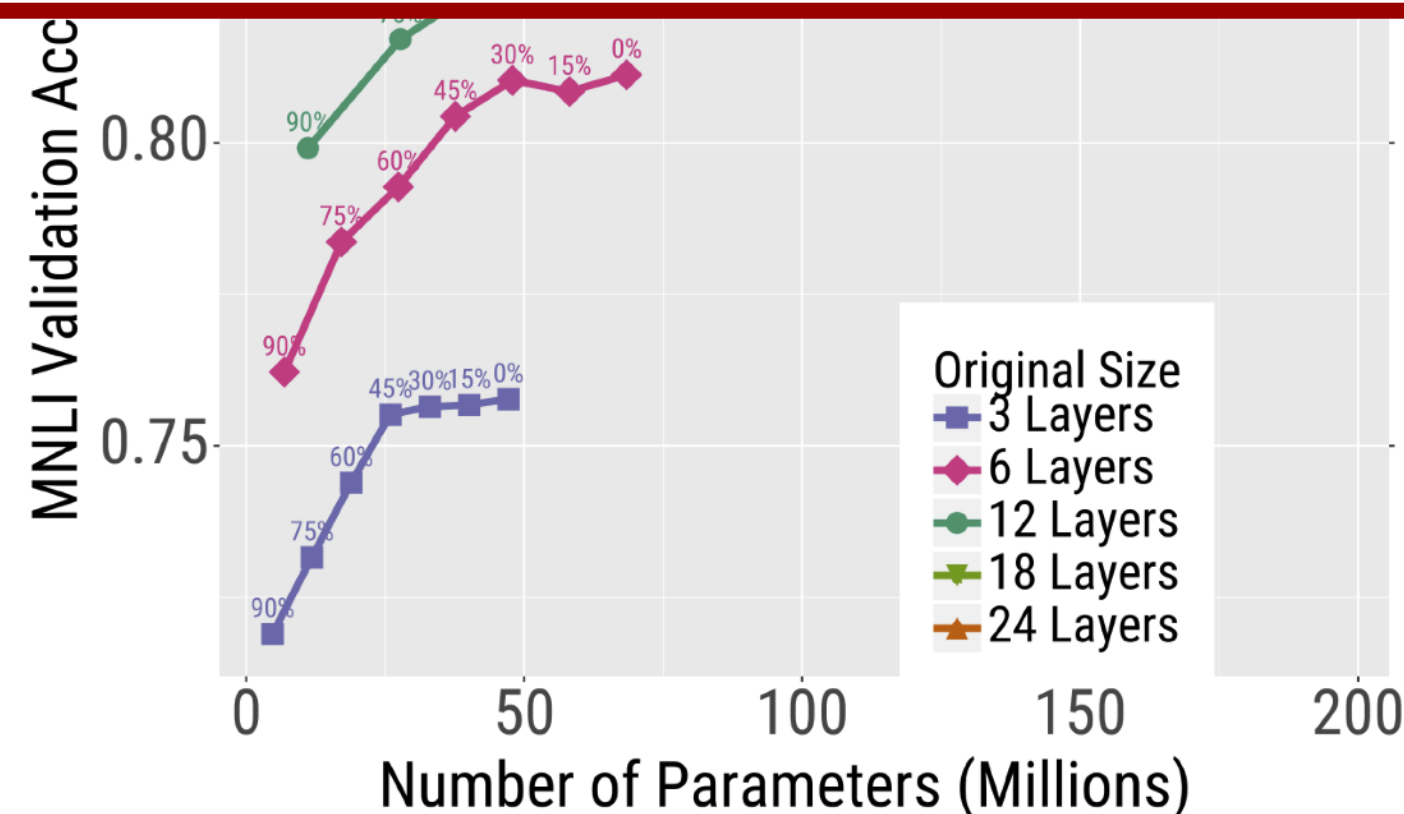
- Large models are more robust to compression techniques than small models
- For given test-time constraints (e.g., inference time, #parameter)
 - heavily compressed, large models > small models
- Comparing downstream task performance for discussed scenarios



Train Large, then Compress!

- Large models are more robust to compression techniques than small models
- For given test-time constraints (e.g., inference time, #parameter)
 - heavily compressed, large models > small models
- Com

Compression improves the model's performance given a test-time budget!



Model Compression

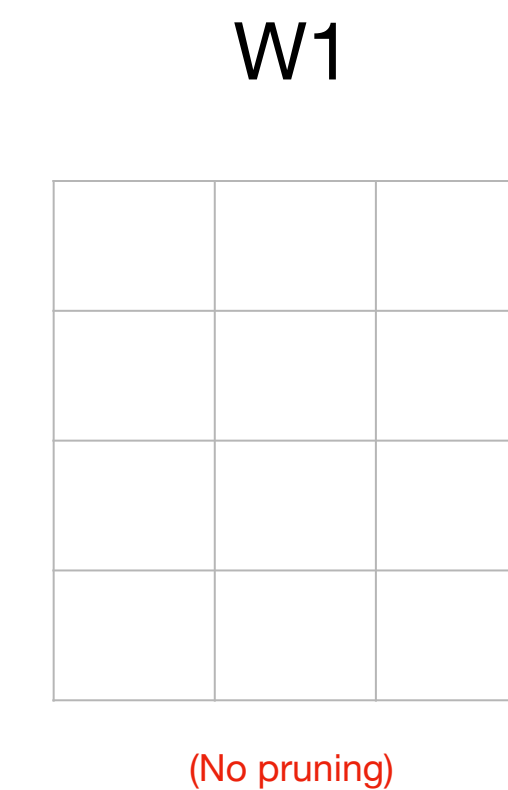
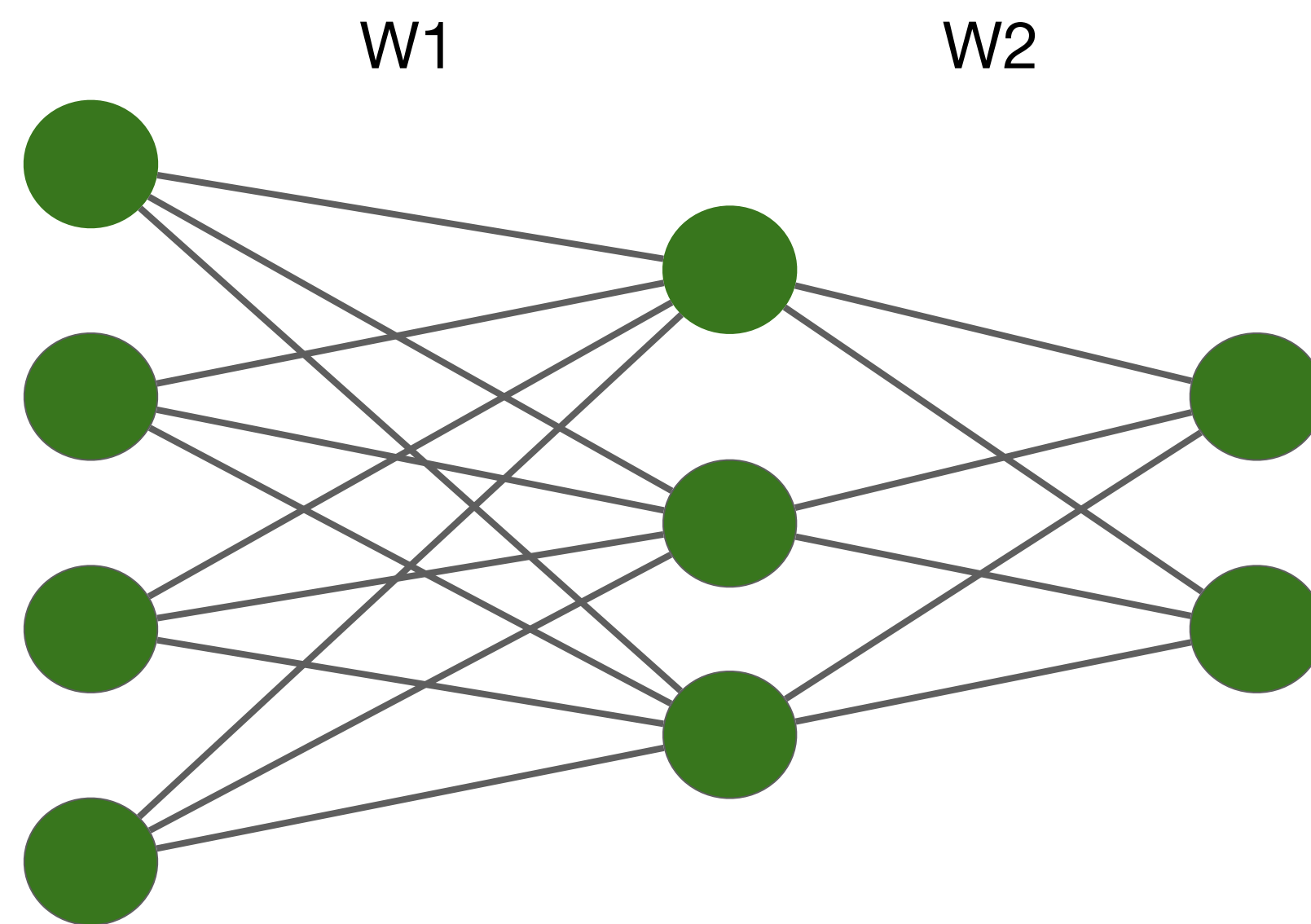
- Pruning
- Quantization
- Knowledge Distillation

Methods Overview

Approach	Improvement on memory footprint	Improvement on inference time
Pruning		
Quantization		
Knowledge distillation		

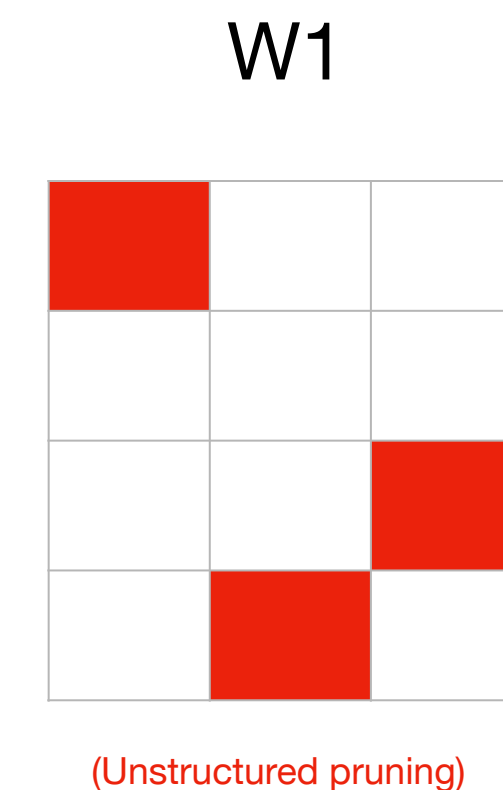
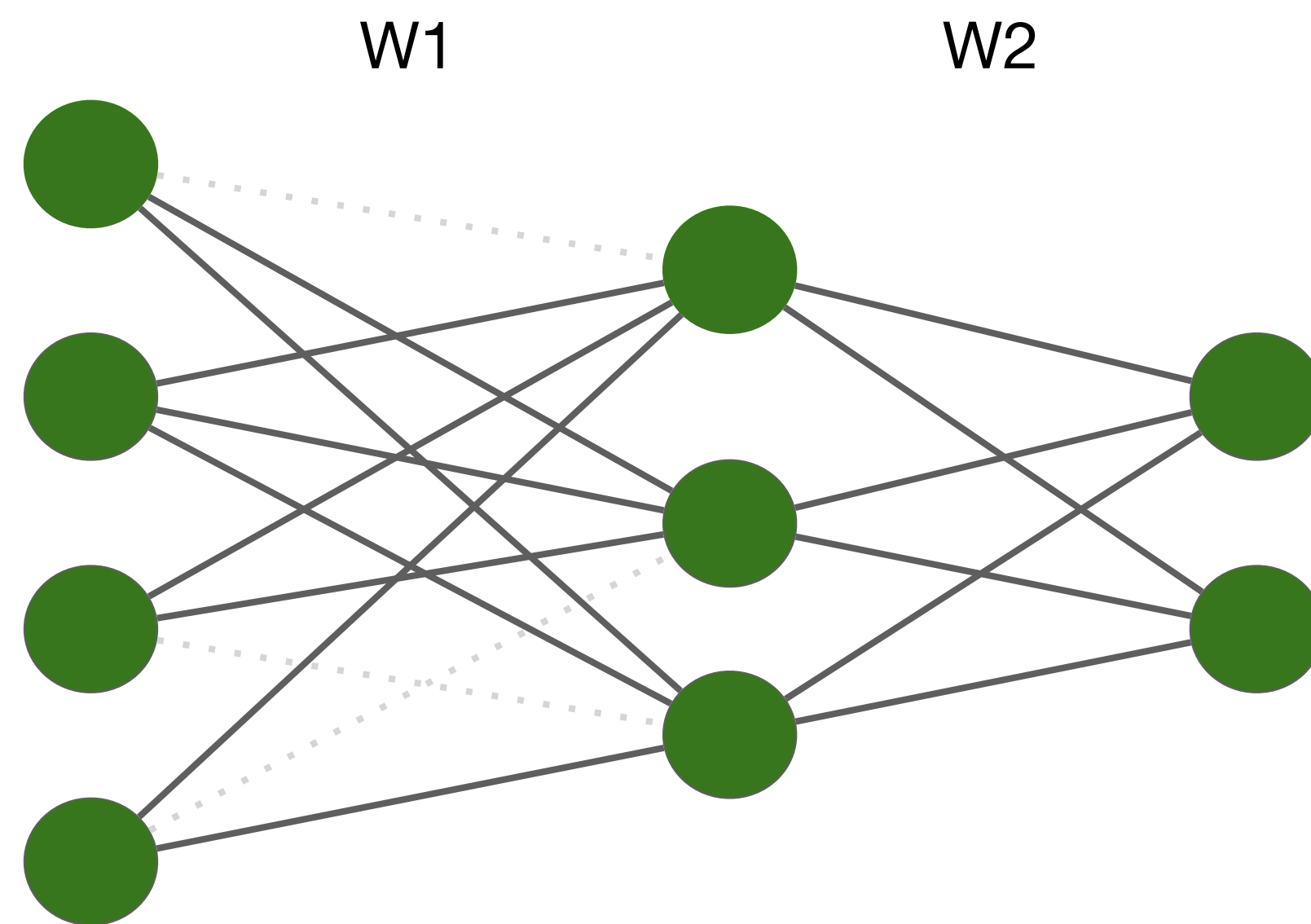
Pruning

- Sparse connectivity inspired by biological neural networks
- Unstructured pruning Vs. structured pruning



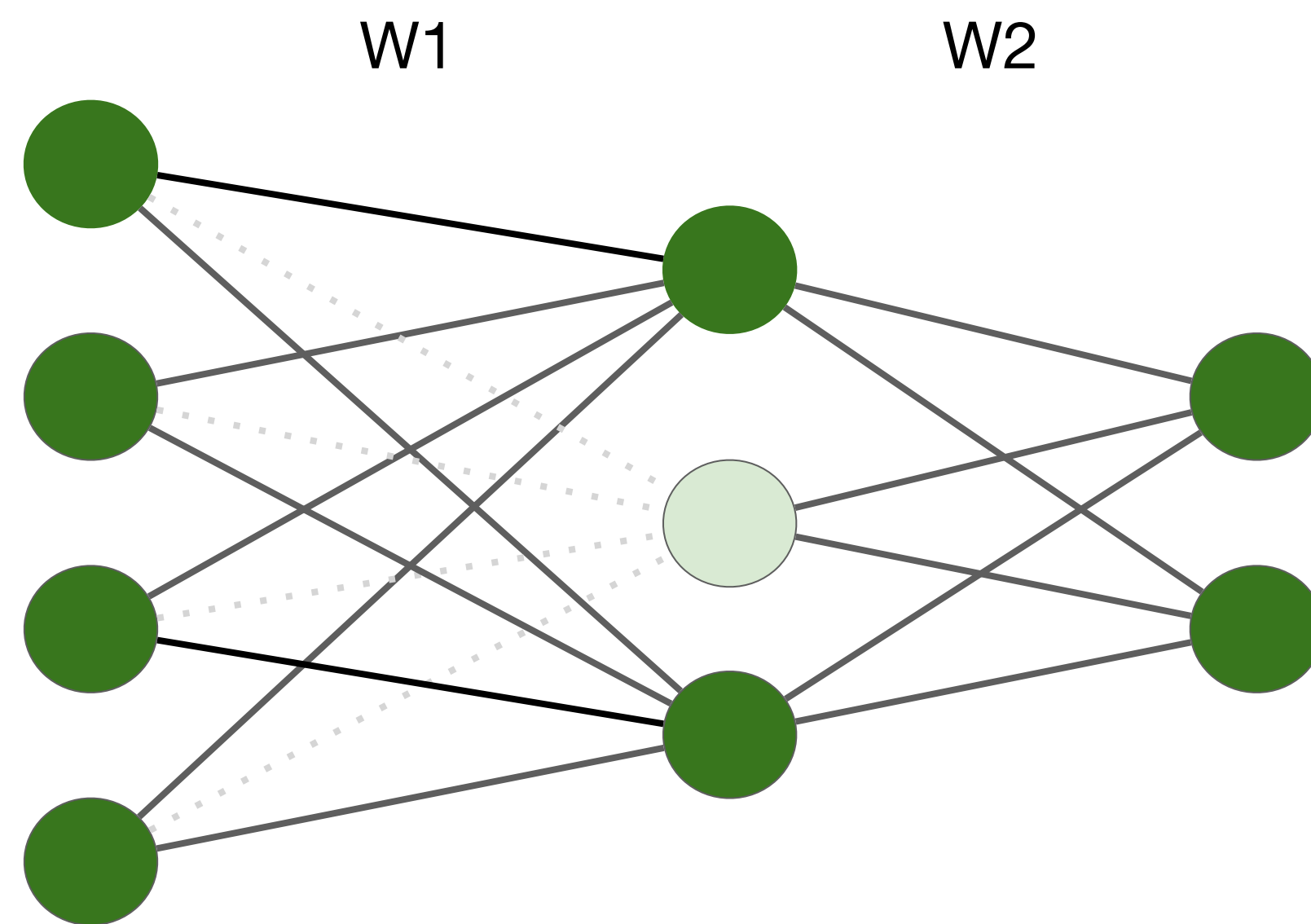
Pruning

- Sparse connectivity inspired by biological neural networks
- **Unstructured pruning (weight-level)** Vs. structured pruning



Pruning

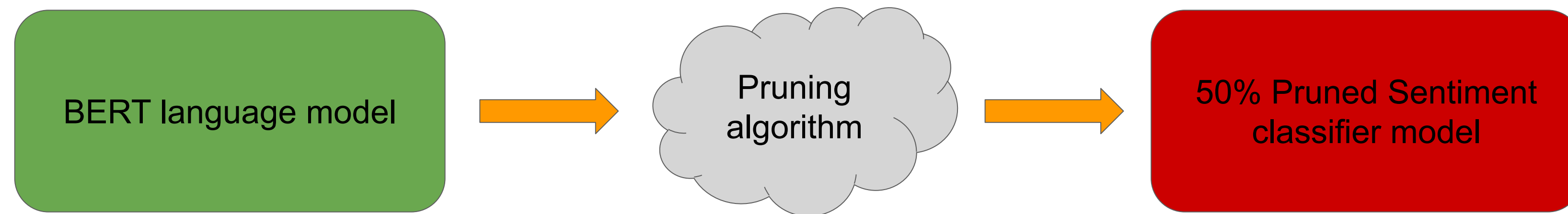
- Sparse connectivity inspired by biological neural networks
- Unstructured pruning Vs. structured pruning (module-level)



How to choose pruned weights?

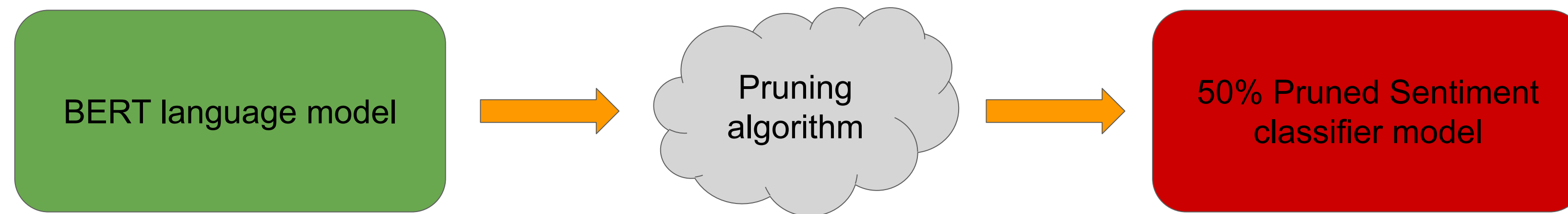
Pruning: case study

- Goal: a BERT-based sentiment classifier model
 - constraints: 50% of weights should be pruned



Pruning: case study

- Goal: a BERT-based sentiment classifier model
 - constraints: 50% of weights should be pruned



- which weights should be **pruned**?

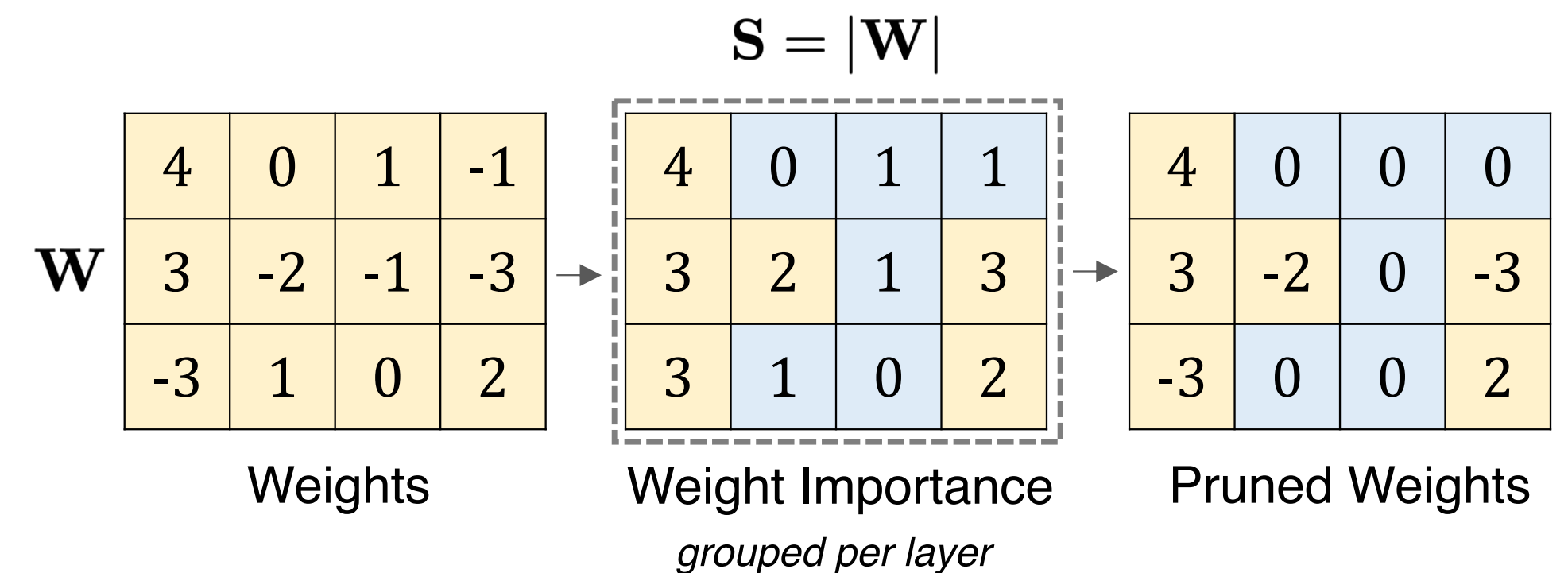
1	2	0.01	3	X1
-0.01	-1	-2	-0.01	X2
-10	-20	0.01	-0.01	X3
				X4

Linear Layer

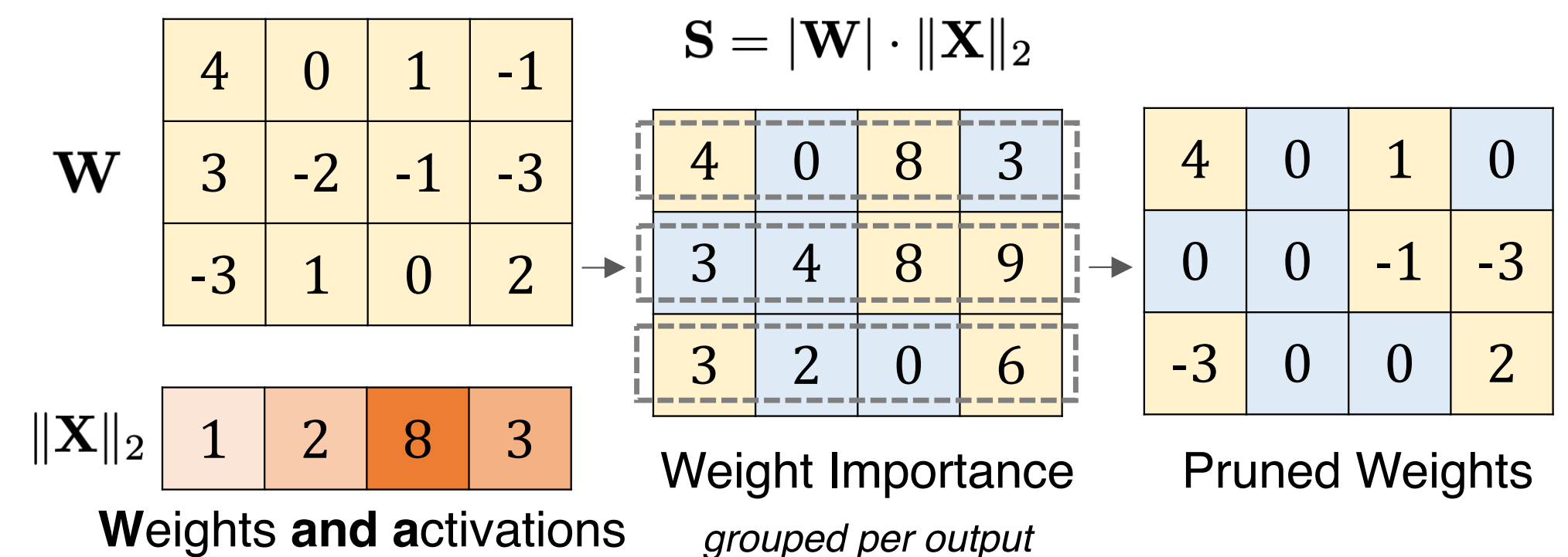
Weight Pruning Methods

- Magnitude pruning
 - Pruning weights with small magnitude
 - Pruning x% at **global** Vs. **module** level
 - **Modern Off-shoot: Wanda**
- Iterative magnitude pruning
 - pruning gradually during training
- Movement pruning
- (Differentiable) masking as a pruning method
 - Example: attention head masking

Magnitude Pruning



Wanda



Frankle, Jonathan, and Michael Carbin. "The lottery ticket hypothesis: Finding sparse, trainable neural networks." *arXiv preprint arXiv:1803.03635* (2018).
 Sanh, Victor, Thomas Wolf, and Alexander Rush. "Movement pruning: Adaptive sparsity by fine-tuning." *Advances in Neural Information Processing Systems* 33 (2020): 20378-20389.
 Zhao, Mengjie, et al. "Masking as an efficient alternative to finetuning for pretrained language models." *arXiv preprint arXiv:2004.12406* (2020).

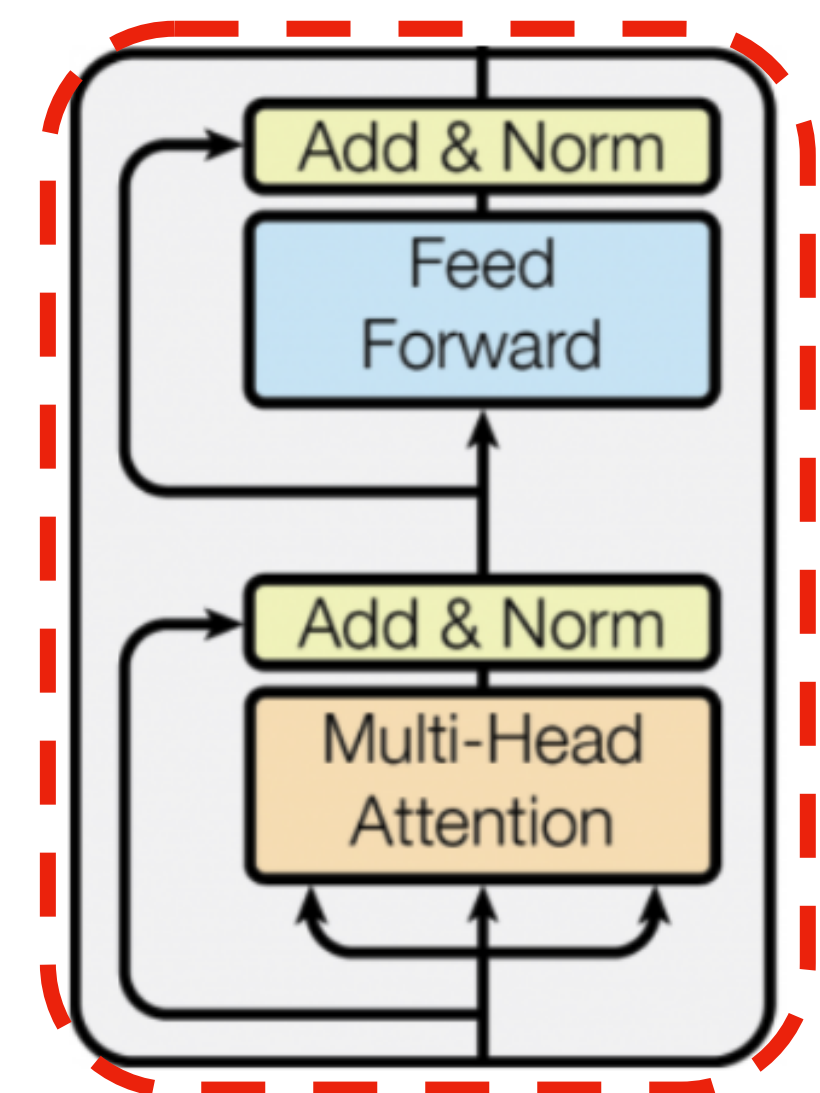
What's a shortcoming of unstructured pruning ?

GPUs don't skip scattered 0s in the weight matrix.

There may be benefits for CPU / edge devices, but won't help much on GPU.

Structured Pruning

- Structured pruning for Transformer language models
 - Pruning neurons
 - Pruning attention heads
 - Pruning sub-layers
 - Example: pruning feed-forward sub-layer
 - Pruning layers
 - Example: pruning the last K layers



Pruning Attention Heads

- How can we prune attention heads?

$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(\text{head}_i) W^O$$

Pruning Attention Heads

- How can we prune attention heads?

$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(\text{head}_i) W^O$$



$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(\boxed{g_i} \cdot \text{head}_i) W^O$$

Pruning Attention Heads

- How can we prune attention heads?

$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(\text{head}_i) W^O$$



$$\text{MultiHead}(Q, K, V) = \text{Concat}_i(\boxed{g_i} \cdot \text{head}_i) W^O$$

- L0 regularization over attention heads' mask parameters

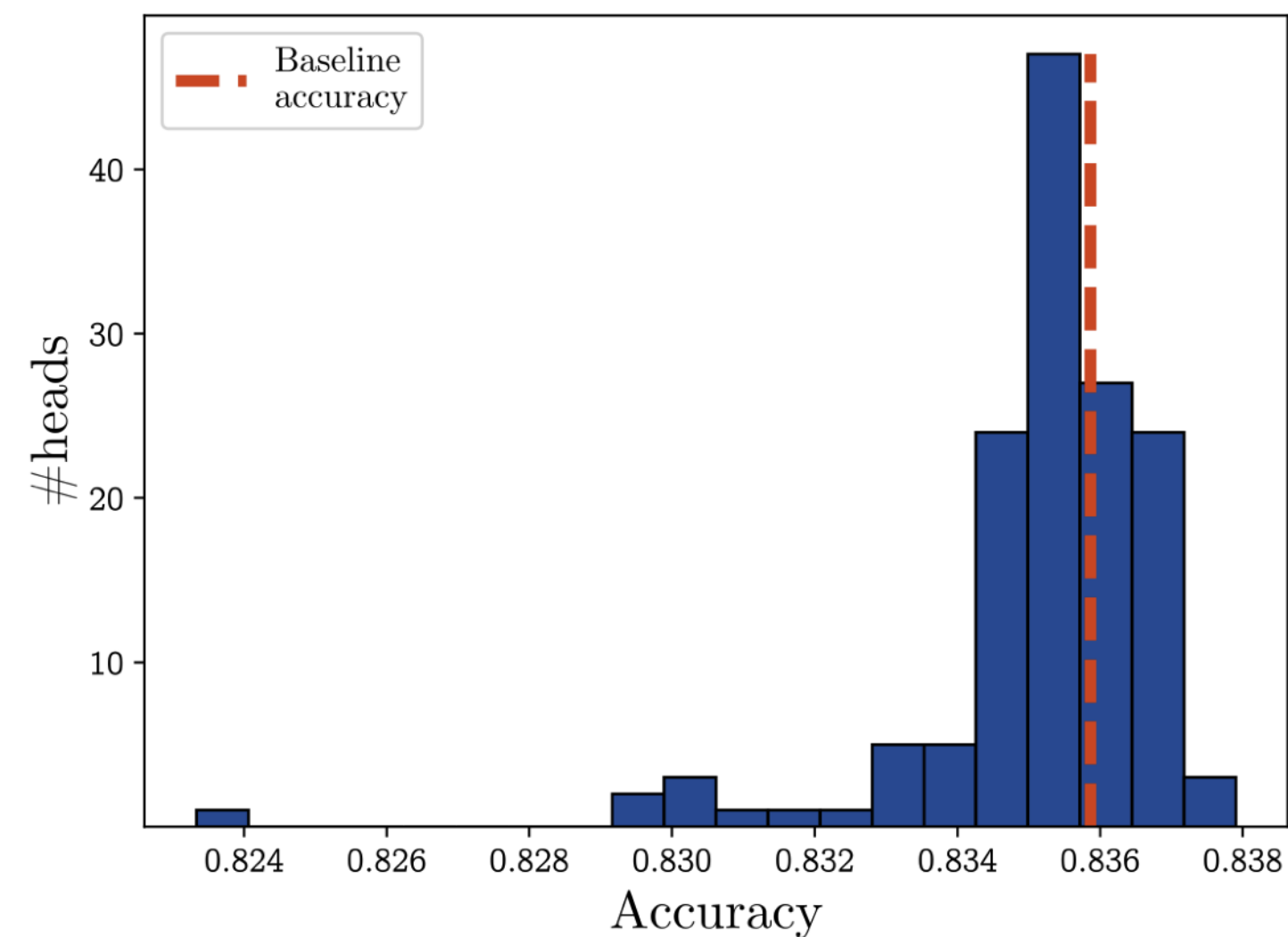
- Example: Translation task

$$L = L_{xent} + \lambda L_C$$

$$\lambda = 0.01$$

Pruning Attention Heads

- Large fraction of Transformer attention heads can be removed at test time!



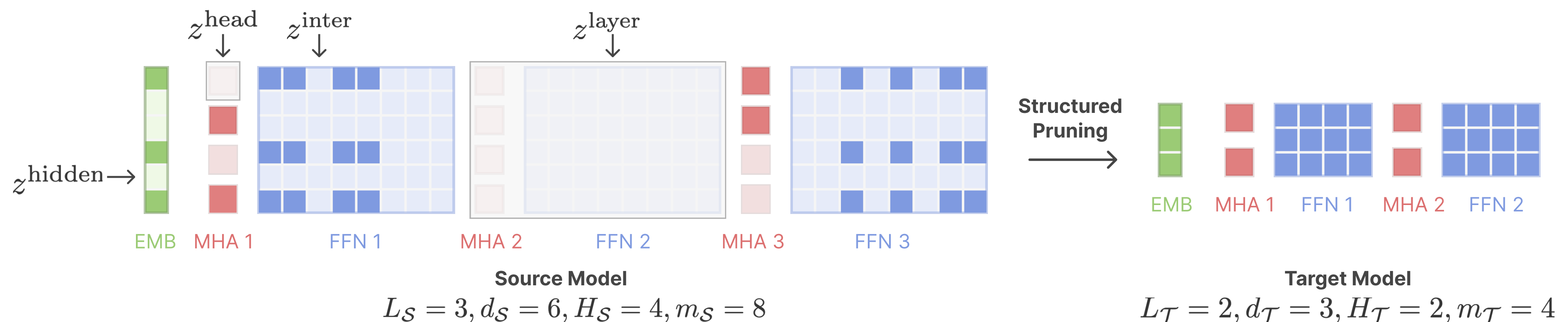
(BERT finetuned on MNLI dataset)

Layer		Layer	
1	-0.01%	7	0.05%
2	0.10%	8	-0.72%
3	-0.14%	9	-0.96%
4	-0.53%	10	0.07%
5	-0.29%	11	-0.19%
6	-0.52%	12	-0.12%

(Delta accuracy by layer when only one head is kept for MNLI BERT model)

Modern Methods: Pruning

- **LLM Pruner** (Ma et al., 2023):
 - Identify coupled structures (e.g., attention head and downstream W^O matrix weights) that can be pruned together)
- **Sheared Llama** (Xia et al., 2023):
 - Prune model using differentiable masking until target architecture is reached.
 - Then, perform continued pretraining on small corpus to recover performance



Methods Overview

Approach	Improvement on memory footprint	Improvement on inference time
Pruning	Depends	Depends
Quantization		
Knowledge distillation		

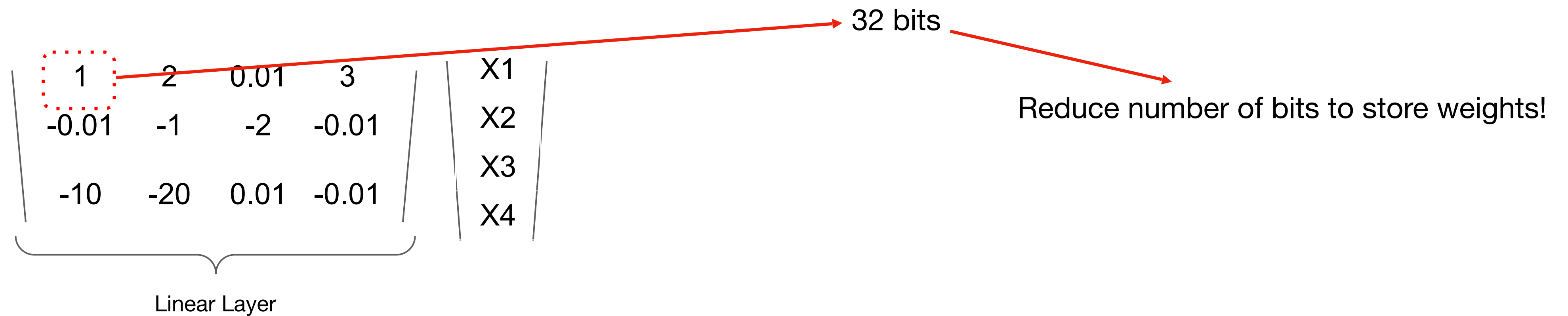
Quantization

- How else can we compress a given neural module?

$$\underbrace{\begin{pmatrix} 1 & 2 & 0.01 & 3 \\ -0.01 & -1 & -2 & -0.01 \\ -10 & -20 & 0.01 & -0.01 \end{pmatrix}}_{\text{Linear Layer}} \begin{pmatrix} X1 \\ X2 \\ X3 \\ X4 \end{pmatrix}$$

Quantization

- How else can we compress a given neural module?



Quantization

- How else can we compress a given neural module?



- Number of parameters remains the same!
 - Improvement in memory footprint + inference time
- Quantization is mostly applied on a **trained** model

Binarized Network

- Essentially using 1 bit per parameter!
- Deterministic Binarization
 - c_1 and c_2 from K-means over the weights
 - c_1 and c_2 **tuned** on downstream task

$$w_b = \begin{cases} c_1 & \text{if } w \geq (c_1 + c_2)/2 \\ c_2 & \text{if } w < (c_1 + c_2)/2 \end{cases}$$

Binarized Network

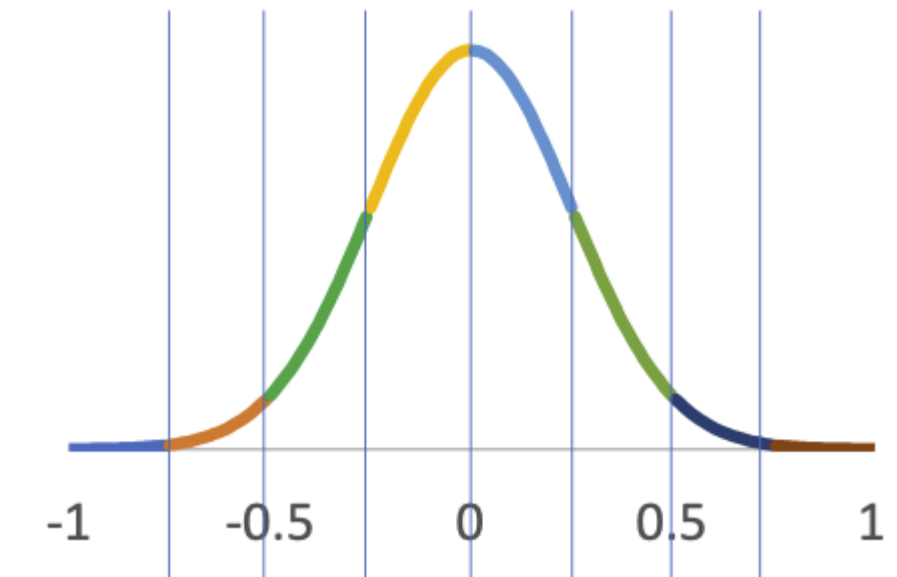
- Essentially using 1 bit per parameter!
- Deterministic Binarization
 - c_1 and c_2 from K-means over the weights
 - c_1 and c_2 **tuned** on downstream task

$$w_b = \begin{cases} c_1 & \text{if } w \geq (c_1 + c_2)/2 \\ c_2 & \text{if } w < (c_1 + c_2)/2 \end{cases}$$

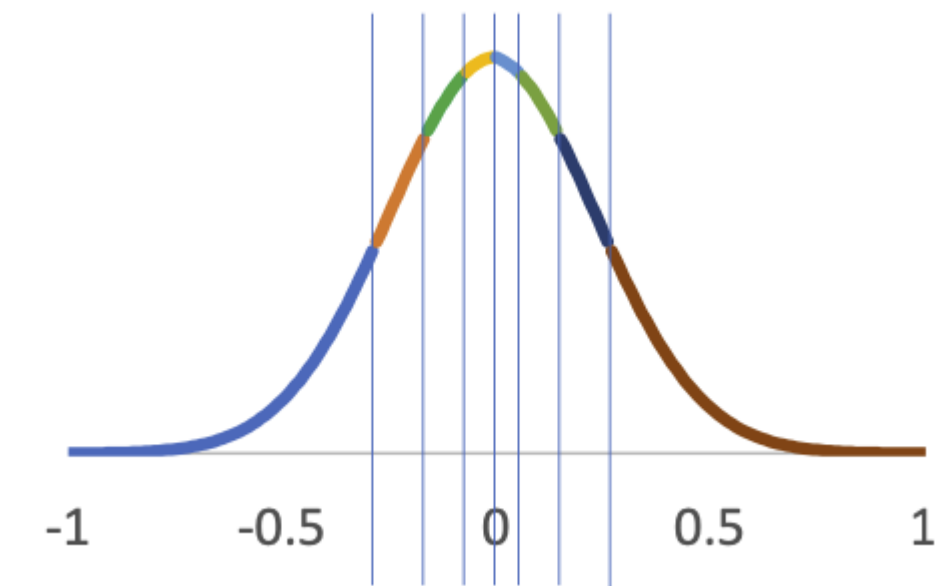
- Question: How can we improve the binarized network performance?

General Quantized Networks

- Uniform Quantization
 - Not necessarily optimal
- Balanced Quantization
 - Better fitted for non-uniform weights!
 - Example: Decide bin boundaries using clustering!



Quantization with 3 bits



Quantization with 3 bits

Methods Overview

Approach	Improvement on memory footprint	Improvement on inference time
Pruning	Depends	Depends
Quantization	Yes	Yes
Knowledge Distillation		

Knowledge Distillation

- Training a smaller **student** network by distilling a large **teacher** model
 - The student's goal is to **imitate** teacher's behavior!
- Can we have the best of the two worlds?
 - Good **performance** of teacher model + **faster** & **parameter**-efficient student model

Knowledge Distillation

- Training a smaller **student** network by distilling a large **teacher** model
 - The student's goal is to **imitate** teacher's behavior!
- Can we have the best of the two worlds?
 - Good **performance** of teacher model + **faster** & **parameter**-efficient student model
- Knowledge distillation Vs. Transfer learning
 - Transfer learning → deals with shared architecture/layers
 - Knowledge distillation → often the student model has a different smaller architecture

How can we distill the teacher's knowledge?

Knowledge Distillation: Intuition

- Consider a 3-class sentiment analysis dataset
 - We pass the following 2 samples to the teacher model to get class probabilities

Sample #1

	Positive	Negative	Neutral
Ground truth	1	0	0
Teacher prob.	0.94	0.01	0.05

Sample #2

	Positive	Negative	Neutral
Ground truth	1	0	0
Teacher prob.	0.67	0.02	0.31

Soft Labels

Knowledge Distillation

- How to leverage **soft** labels for the student model?

- Additional cross-entropy to soft labels (**soft loss**)
- Cross-entropy loss to ground-truth labels → **hard** loss

$$\mathcal{L} = \underbrace{\alpha \cdot \mathcal{L}_{\text{CE}}}_{\text{Hard Loss}} + (1 - \alpha) \cdot \underbrace{\mathcal{L}_{\text{distill}}}_{\text{Soft Loss}}$$

- Teacher making confident prediction for easy downstream tasks

- Solution: increase softmax temperature to get suitably soft targets!

Knowledge Distillation

- How to leverage **soft** labels for the student model?

- Additional cross-entropy to soft labels (**soft loss**)
- Cross-entropy loss to ground-truth labels → **hard** loss

$$\mathcal{L} = \underbrace{\alpha \cdot \mathcal{L}_{\text{CE}}}_{\text{Hard Loss}} + (1 - \alpha) \cdot \underbrace{\mathcal{L}_{\text{distill}}}_{\text{Soft Loss}}$$

- Teacher making confident prediction for easy downstream tasks

- Solution: increase softmax temperature to get suitably soft targets!

#	Model	SST-2	QQP	MNLI-m	MNLI-mm
		Acc	F ₁ /Acc	Acc	Acc
1	BERT _{LARGE} (Devlin et al., 2018)	94.9	72.1/89.3	86.7	85.9
2	BERT _{BASE} (Devlin et al., 2018)	93.5	71.2/89.2	84.6	83.4
3	OpenAI GPT (Radford et al., 2018)	91.3	70.3/88.5	82.1	81.4
4	BERT ELMo baseline (Devlin et al., 2018)	90.4	64.8/84.7	76.4	76.1
5	GLUE ELMo baseline (Wang et al., 2018)	90.4	63.1/84.3	74.1	74.5
6	Distilled BiLSTM _{SOFT}	90.7	68.2/88.1	73.0	72.6
7	BiLSTM (our implementation)	86.7	63.7/86.2	68.7	68.3

Case Study: distilBERT

- 6-layer student model distilled from BERT-base (i.e., teacher)
 - Initialize the student from the teacher by taking one layer out of two

Case Study: distilBERT

- 6-layer student model distilled from BERT-base (i.e., teacher)
 - Initialize the student from the teacher by taking one layer out of two
- Distillation on MLM loss
 - Improving LM generalization

I absolutely [MASK] natural language processing field. 

Case Study: distilBERT

- 6-layer student model distilled from BERT-base (i.e., teacher)
 - Initialize the student from the teacher by taking one layer out of two
- Distillation on MLM loss
 - Improving LM generalization

I absolutely [MASK] natural language processing field. → BERT-base

0.241	I absolutely hate natural language processing field.
0.154	I absolutely love natural language processing field.
0.045	I absolutely need natural language processing field.
0.041	I absolutely mean natural language processing field.
0.040	I absolutely missed natural language processing field.
0.034	I absolutely hated natural language processing field.
0.032	I absolutely understand natural language processing field.
0.024	I absolutely loved natural language processing field.
0.023	I absolutely like natural language processing field.
0.020	I absolutely miss natural language processing field.

Case Study: distilBERT

- 6-layer student model distilled from BERT-base (i.e., teacher)
 - Initialize the student from the teacher by taking one layer out of two
- Distillation on MLM loss
 - Improving LM generalization

I absolutely [MASK] natural language processing field.

BERT-base

0.241	I absolutely hate natural language processing field.
0.154	I absolutely love natural language processing field.
0.045	I absolutely need natural language processing field.
0.041	I absolutely mean natural language processing field.
0.040	I absolutely missed natural language processing field.
0.034	I absolutely hated natural language processing field.
0.032	I absolutely understand natural language processing field.
0.024	I absolutely loved natural language processing field.
0.023	I absolutely like natural language processing field.
0.020	I absolutely miss natural language processing field.

- **Proposed Loss: MLM + distilling BERT**
MLM + distilling hidden states

Case Study: distilBERT

- 6-layer student model distilled from BERT-base (i.e., teacher)
 - Initialize the student from the teacher by taking one layer out of two
- Distillation on MLM loss
 - Improving LM generalization

I absolutely [MASK] natural language processing field.

BERT-base

0.241 I absolutely hate natural language processing field.
0.154 I absolutely love natural language processing field.
0.045 I absolutely need natural language processing field.
0.041 I absolutely mean natural language processing field.
0.040 I absolutely missed natural language processing field.
0.034 I absolutely hated natural language processing field.
0.032 I absolutely understand natural language processing field.
0.024 I absolutely loved natural language processing field.
0.023 I absolutely like natural language processing field.
0.020 I absolutely miss natural language processing field.

- **Proposed Loss: MLM + distilling BERT**

MLM + distilling hidden states

- **Competitive performance to the teacher**

Model	IMDb (acc.)	SQuAD (EM/F1)
BERT-base	93.46	81.2/88.5
DistilBERT	92.82	77.7/85.8
DistilBERT (D)	-	79.1/86.9

Methods Overview

Approach	Improvement on memory footprint	Improvement on inference time
Pruning	Depends	Depends
Quantization	Yes	Yes
Knowledge distillation	Yes	Yes

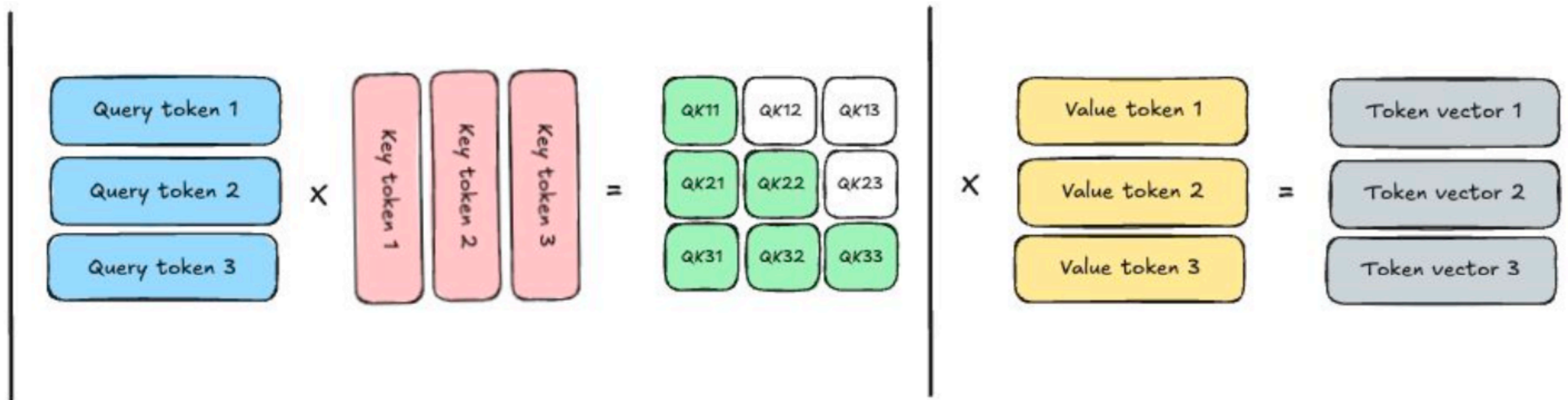
Inference-time Efficiency

- The autoregressive nature of text generation creates a large overhead on models as they generate tokens
 - At every step, the model must compute multiple layers of attention and FFN operation
- **Solutions:**
 - KV Cache
 - Speculative Decoding

KV Cache

- **Problem:** The autoregressive nature of text generation creates a large overhead on models as they generate tokens
 - At every step, the model must compute multiple layers of attention and FFN operations
 - **At every step, the model repeats all the computations from the previously generated tokens**
 - But, since the model is unidirectional, aren't the attention weights between all previous tokens exactly the same ?
- **Can we cache the results of these operations ?**

KV Cache



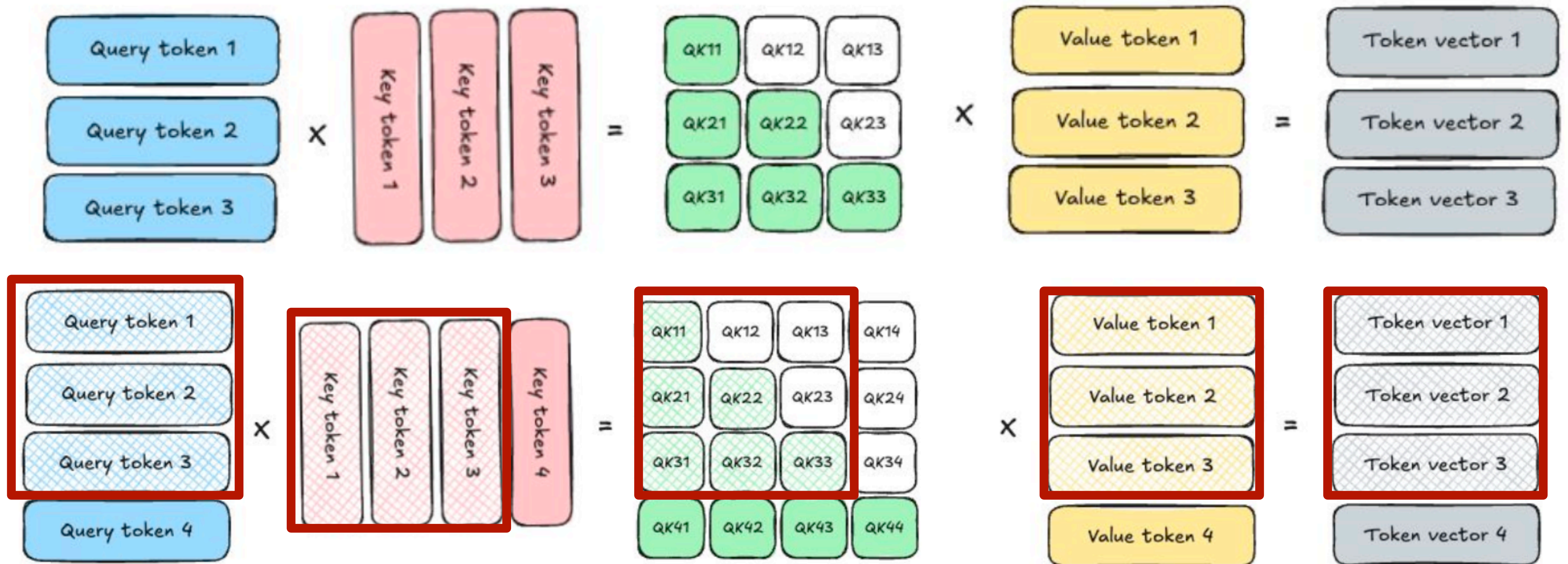
Standard attention: Dot product of queries and keys compute attention weights, which scale value vectors. Equations adapted from Week 3:

$$\mathbf{qk}_{st} = \frac{(\mathbf{W}^Q \mathbf{Q}_s)^T (\mathbf{W}_t^K \mathbf{K})}{\sqrt{d}}$$

$$\alpha_{st} = \mathbf{softmax}(\mathbf{qk}_{st})$$

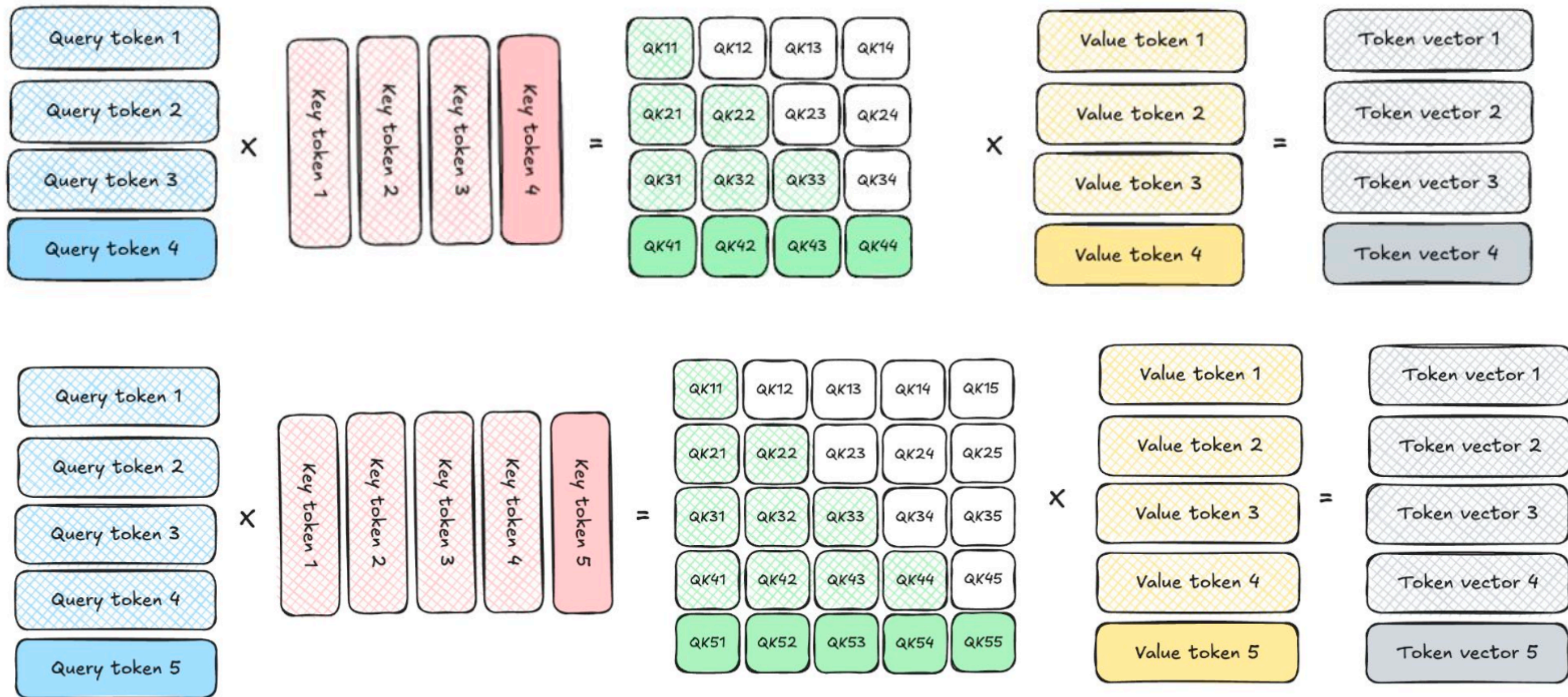
$$\mathbf{M}_s = \mathbf{W}^O \alpha_{st} (\mathbf{W}^V \mathbf{V}_t)$$

KV Cache

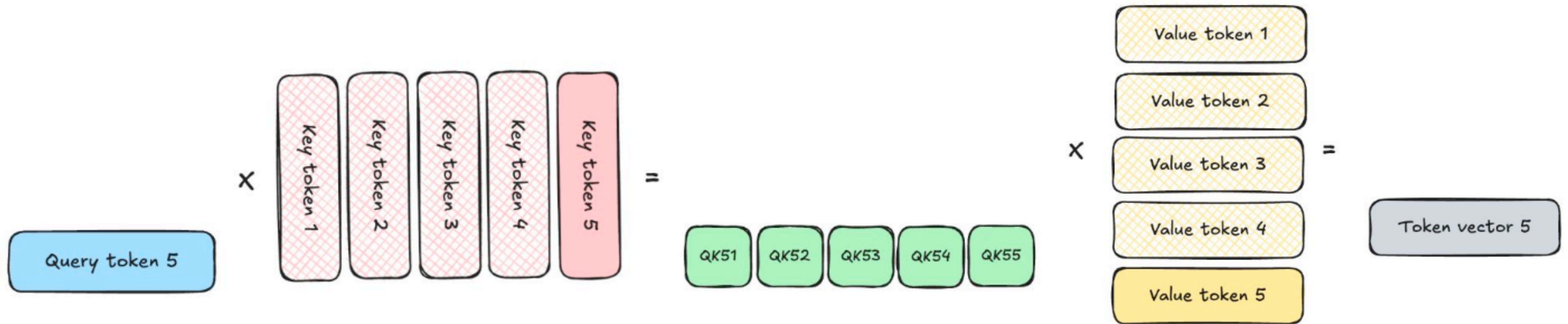


All of these are the same as when the previous token was generated !

KV Cache



KV Cache



KV Cache: after computing key and value vectors, cache the vector results so they don't have to be recomputed. Used in every production LLM deployment.

Tradeoff: Compute cost grows linearly with generation length Cache takes memory (enormous amounts of it).

Solution: Many new compression methods exist for the KV cache (cache eviction, quantization, new attention variants)

Methods Overview

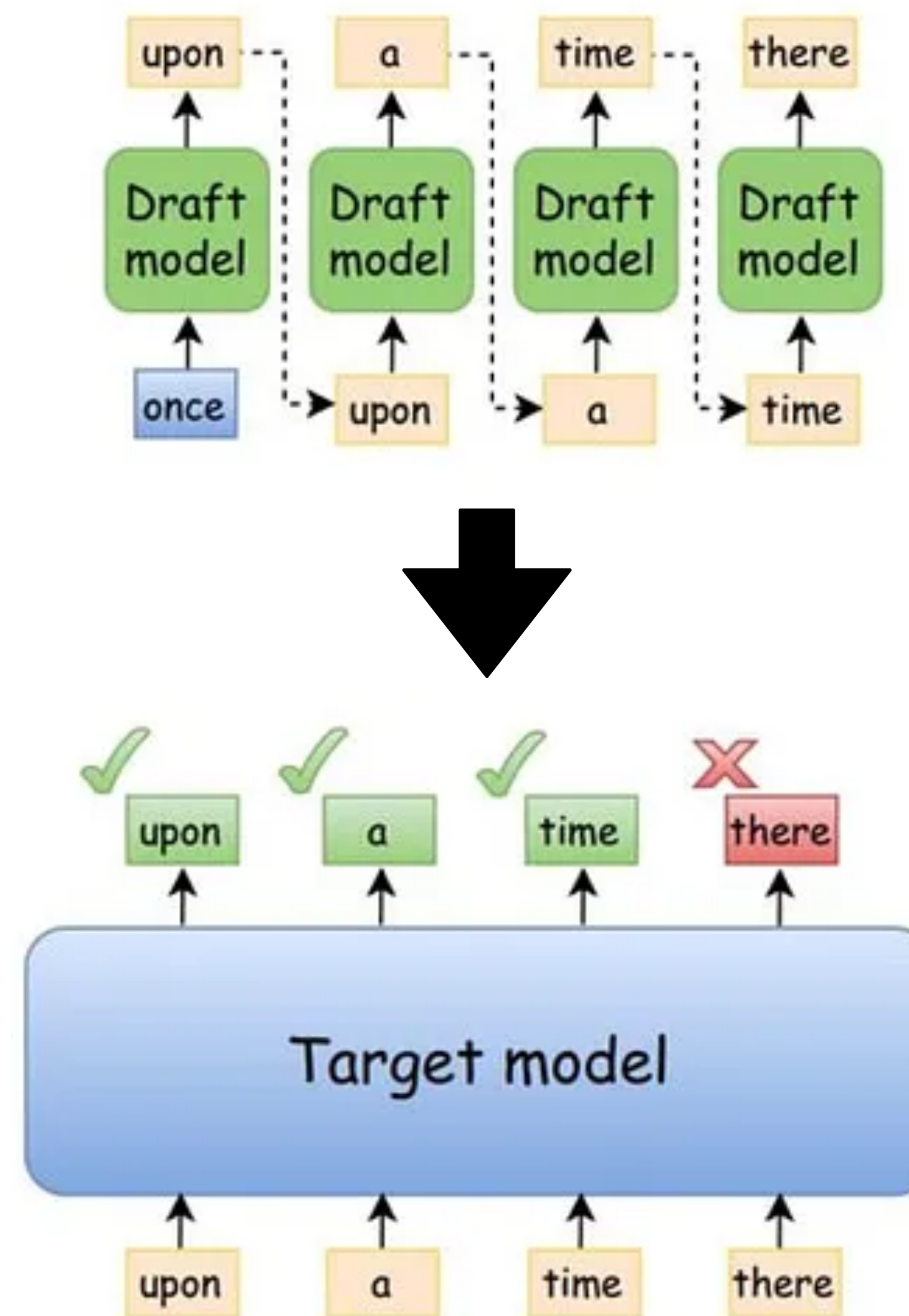
Approach	Improvement on memory footprint	Improvement on inference time
Pruning	Depends	Depends
Quantization	Yes	Yes
Knowledge distillation	Yes	Yes
KV Cache	No, way worse	Yes
Speculative Decoding		

Speculative Decoding

- Large models have a much higher decoding cost during inference
 - KV cache saves latency cost, but still full forward pass for every token generated !
- **Solution:** use a small model to generate candidate sequences, and **verify** that the large model would have also generated the same sequences
 - smaller model performs full forward pass for every token generated, and larger model only does forward pass in verification steps.

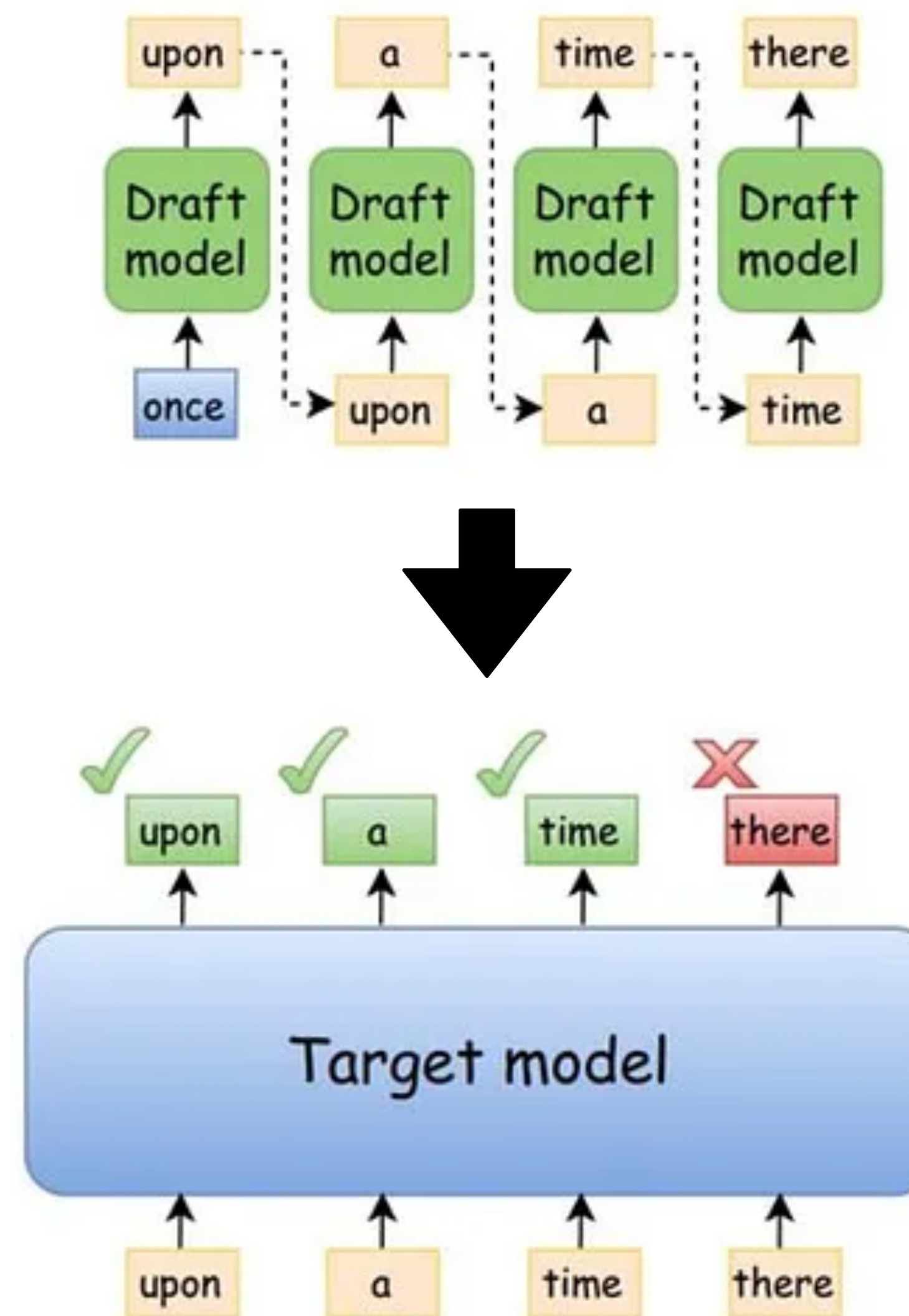
Speculative Decoding

- Step 1: Generate from smaller draft model
 - Typically set window k as a hyper parameter of the number of tokens to generated !
- Step 2: Verify generated tokens in parallel using larger target model
 - If generated tokens are “in distribution” of target model, keep the generated tokens.
 - If not, reject draft tokens and decode from target model at first generated token that is not “in distribution”



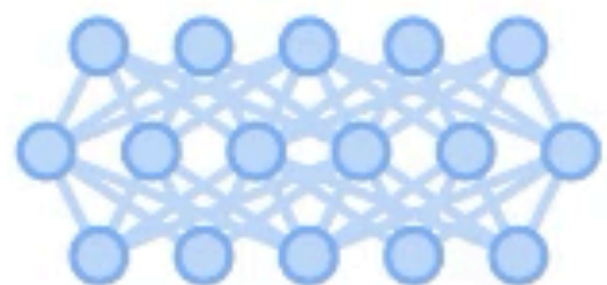
Speculative Decoding

- What does “in distribution” mean?
 - **Greedy decoding**: same max-probability token
 - **Sampling**: probability of token within some bound of max-probability token of the target model
 - More details: <https://arxiv.org/abs/2211.17192>
- Considerations
 - Large k : many rejections (draft model wasted computations)
 - Small k : target model verifies more often (large computation)



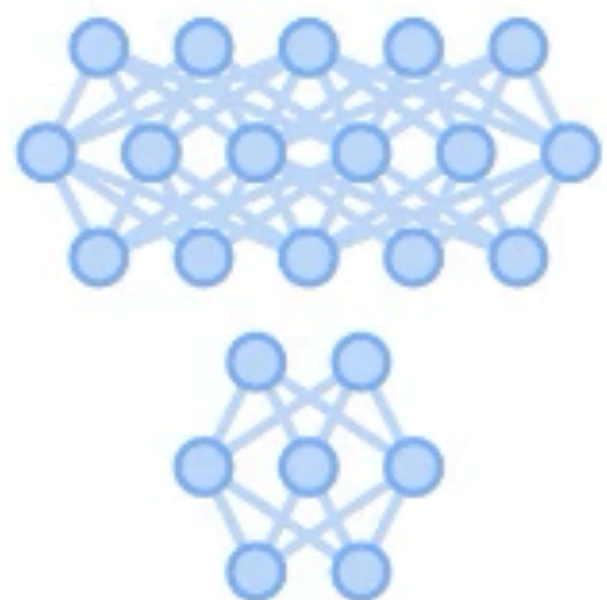
Speculative Decoding

WITHOUT SPECULATIVE DECODING



My favorite thing about fall

WITH SPECULATIVE DECODING



My favorite thing about fall

Methods Overview

Approach	Improvement on memory footprint	Improvement on inference time
Pruning	Depends	Depends
Quantization	Yes	Yes
Knowledge distillation	Yes	Yes

KV Cache	No, way worse	Yes
Speculative Decoding	No	Yes

Recap

- Compression leads to improving:
 - Number of parameters
 - Inference time
- Different compression techniques
 - Pruning, quantization, knowledge distillation
- Size-performance trade-off
 - Heavily compressed large models > lightly compressed small models
- Inference-time efficiency trades off memory for latency
 - KV Cache
 - Speculative Decoding